

Proxy | Provide a surrogate or placeholder for another object to control access to it.

 DIVE 01 · 10 km
Airplane porthole
The big picture & contours

→

 DIVE 02 · 1 km
Train window
Theses, examples, terms

→

 DIVE 03 · 100 m
Park walk
How the pieces relate

→

 DIVE 04 · 1 cm
Microscope
Exact wording & gotchas

 Airplane porthole

DIVE 01 · ORIENTATION

▼ 10 km

A surrogate with the same interface as its subject — it intercepts every call so it can guard, defer, cache or relay before the real work.

THE CONTOURS

• Same interface as subject

callers can't tell the proxy from the real object

• Intercepts access

runs logic before, after or instead of forwarding

• Many flavors

virtual, remote, protection, smart — one shape, many jobs

USE IT WHEN — GoF applicability

 Need a smarter reference

than a plain pointer — lazy load, access control, remoting, caching

 Defer expensive work

create or fetch the real subject only when it's actually used

 Structural family: Proxy controls access · Decorator adds behavior · Adapter changes interface · Facade simplifies many. Proxy & Decorator share a shape; intent differs.

↓ DIVE DEEPER

 Train window

DIVE 02 · KEY OBJECTS

▼ 1 km

THE THREE PARTS

Subject

the common interface shared by RealSubject and Proxy

RealSubject

the real object the proxy represents and forwards to

Proxy

holds a ref, controls access, then delegates to RealSubject

IN THE WILD

 ORM lazy loading

Hibernate / TypeORM proxy entities, hit the DB on first access

 RPC / gRPC stubs

a local proxy marshals calls to a remote object

 JS Proxy & reflection

traps get / set — powers Vue & MobX reactivity, validation

 API gateway / CDN

caching & protection proxies sitting in front of origin

<> Client → Proxy.request() → guards / defers / caches, then forwards to RealSubject.request().

GoF intent: “Provide a surrogate or placeholder for another object to control access to it.”

↓ DIVE DEEPER

 Park walk

DIVE 03 · CONNECTIONS

▼ 100 m

Client

calls request()

→

Proxy

controls access

→

guard / defer / cache

policy runs

→

✓ RealSubject

real work runs

WHY IT FOLLOWS

→ Same interface → the proxy is a drop-in for the real subject, transparently

→ Holds the reference → it decides whether, when and how to forward each call

→ Indirection → a hook for laziness, security or remoting without touching the subject

➤ Same wrapper shape as Decorator (adds behavior) and Adapter (changes interface) — Proxy controls access. A remote proxy underpins RPC; a virtual proxy is lazy init; copy-on-write is a smart-reference proxy.

STRUCTURE · UML

Proxy

- real: RealSubject

+ request(): void

checkAccess()

Proxy.request() may create, guard or cache, then forwards to real.request(); the client sees only Subject.

WHAT IT BUYS YOU

✓ Virtual proxy — defers creation / loading of an expensive subject until needed

✓ Remote proxy — hides that the real object lives in another address space

✓ Protection proxy — checks access rights before forwarding the call

✓ Smart reference — adds ref-counting, locking, caching or logging around access

✓ Transparent to clients — all of this happens behind the subject's own interface

↓ DIVE DEEPER

 Microscope

DIVE 04 · DETAILS

▼ 1 cm

Definition: “Provide a surrogate or placeholder for another object to control access to it.”

— GoF, “Design Patterns”, 1994

THE CODE

```
interface Image { draw(): void }
class RealImage implements Image {
  constructor(f) { this.load(f); } // costly
  draw() {}
}
class ProxyImage implements Image {
  constructor(private f) {}
  draw() { // lazy-create on first use
    (this.img ??= new RealImage(this.f)).draw();
  }
}
```

VARIANTS

Virtual proxy

lazy-create / load an expensive subject on demand

Remote proxy

a local stand-in for an object in another process

Protection proxy

enforces access control before forwarding

Smart reference

ref-count, lock, cache or log around access

Caching proxy

memoizes the results of expensive calls

Copy-on-write

shares until a write, then clones — a smart ref

COMPARE THE ALTERNATIVES

Property	Proxy	Decorator	Adapter	Facade
Same interface as target	✓	✓	✗	✗
Controls access	✓	✗	✗	~
Adds new behavior	~	✓	✗	✗
Changes the interface	✗	✗	✓	✓
Client sees the difference	✗	✗	~	✓

Proxy gates access · Decorator adds behavior · Adapter converts · Facade simplifies many.

INTERVIEW PITFALLS

Q “Proxy vs Decorator?”

Same wrapping structure; intent differs — Proxy controls access (lazy, remote, guard), Decorator adds responsibilities.

Q “Name the four proxy kinds.”

Virtual (lazy), Remote (cross-process), Protection (access control), Smart reference (ref-count / cache / lock).

Q “How does ORM lazy loading use it?”

Entities are virtual proxies; touching a lazy field triggers the DB fetch behind the same interface.

Q “What is a JS Proxy?”

A language-level proxy whose traps (get / set / has) intercept operations — it powers Vue & MobX reactivity.

NUANCES & GOTCHAS

• Proxy vs Decorator — identical shape; Proxy controls access, Decorator adds behavior

• Keep it transparent — a proxy must honor the subject's full interface and semantics

• Hidden latency / failure — a remote proxy can block or throw where a local call wouldn't

• Lazy means surprises — first-touch creation moves cost and errors to odd places

• Identity & equality — proxy !== realSubject; instanceof and == can mislead

• JS Proxy traps are subtle — invariants, this-binding and revoked proxies bite

WATCH OUT

 Anti-pattern:

Skip it when you don't need to intercept access — a plain reference is simpler. And beware hiding network calls behind a local-looking interface; explicit async is often clearer than a transparent remote proxy.

 Modern take:

the JS Proxy object and ORM lazy entities are everywhere; reactivity systems (Vue 3, MobX, Solid) are smart / protection proxies trapping property access.

• Vasyi Krupka · Senior Fullstack Engineer · learn-by-diving series

Source: GoF “Design Patterns” (1994) v1